Proyectos reproducibles de ciencia de datos

Una guía práctica

Tabla de contenidos

1 Introducción **2**
- 1.1 Dos dimensiones complementarias de la reproducibilidad . . . 2
- 1.2 Relación con CRISP-DM . . . 3

2 Organización de un proyecto de investigación reproducible **3**
- 2.1 El repositorio como unidad de reproducibilidad . . . 3
- 2.2 Una estructura mínima de directorios . . . 4
- 2.3 Archivos fuente y artefactos generados . . . 5
- 2.4 Nombres y rutas . . . 5

3 Cuadernos computacionales **6**
- 3.1 La función de los cuadernos de Quarto . . . 6
- 3.2 Una responsabilidad analítica por cuaderno . . . 6
- 3.3 Anatomía estándar de un cuaderno . . . 6
- 3.4 Ejecución independiente y dependencias explícitas . . . 7
- 3.5 Transparencia y reproducibilidad computacionales . . . 7

4 Informes científicos **8**
- 4.1 Los informes como comunicación científica . . . 8
- 4.2 Informes y cuadernos como registros complementarios . . . 8
- 4.3 Relación con los hitos de entrega de informes . . . 9

5 Flujos de trabajo modulares y automatizados **9**
- 5.1 Modularidad analítica y modularidad del software . . . 9
- 5.2 Funciones reutilizables de R y `src/` . . . 9
- 5.3 Scripts ejecutables y `scripts/` . . . 10
- 5.4 Del análisis interactivo a un proceso automatizado . . . 10
- 5.5 Verificaciones de reproducibilidad . . . 11

6 Principios finales 11

1. Introducción

Un proyecto de investigación en ciencia de datos implica más que aplicar métodos estadísticos a un conjunto de datos. Requiere formular una pregunta significativa, identificar y evaluar información pertinente, preparar los datos para el análisis, interpretar los resultados y comunicar los hallazgos. Un proyecto reproducible pone a disposición de otras personas tanto el razonamiento como los cálculos que sustentan estas actividades.

Esta guía presenta un marco organizativo para proyectos de ciencia de datos orientados a la investigación que utilizan **R**, **Quarto** y un repositorio con control de versiones. Su principal interés es cómo organizar los documentos y los componentes computacionales de un proyecto para que la investigación pueda comprenderse, evaluarse y reproducirse. Es una guía de consulta, no una secuencia prescrita de tareas de un curso.

1.1. Dos dimensiones complementarias de la reproducibilidad

La **reproducibilidad computacional** es la capacidad de obtener los mismos resultados computacionales al ejecutar los procedimientos documentados con los mismos insumos y un entorno de software suficientemente equivalente. Depende de la disponibilidad de datos, código, dependencias, configuración e instrucciones de ejecución. Cuando interviene la aleatoriedad, también deben controlarse las semillas y otras configuraciones pertinentes; algunos cálculos aun así pueden variar entre plataformas o versiones de software.

La **reproducibilidad metodológica**, tal como se utiliza en esta guía, consiste en hacer suficientemente explícitos el diseño de la investigación y las decisiones analíticas para que otro investigador pueda comprender, examinar críticamente y aplicar los mismos procedimientos. Esto incluye la formulación de las preguntas de investigación, la elección de observaciones y variables, el tratamiento de los datos faltantes, la selección de métodos estadísticos y la interpretación de los resultados. Este término se utiliza aquí como una distinción organizativa de carácter práctico; la terminología varía entre campos de investigación.

Estas dimensiones son complementarias, pero ninguna garantiza la otra. Un script puede reproducir un resultado numérico sin explicar por qué el cálculo era apropiado. A la inversa, un informe cuidadosamente justificado puede ser imposible de reproducir computacionalmente si su código o sus datos no están disponibles.

Dimensión	Pregunta orientadora	Evidencia principal
Reproducibilidad computacional	¿Pueden ejecutarse de nuevo los cálculos?	Datos, código ejecutable, dependencias, configuración, instrucciones
Reproducibilidad metodológica	¿Pueden comprenderse y evaluarse las decisiones de investigación?	Preguntas, supuestos, criterios de decisión, justificaciones, limitaciones

La reproducibilidad es distinta de la **replicación**, que comúnmente se refiere a investigar si un resultado puede obtenerse de nuevo utilizando datos nuevos o un estudio realizado de manera independiente. El uso de estos términos difiere entre disciplinas.

1.2. Relación con CRISP-DM

El Proceso Estándar Intersectorial para la Minería de Datos (CRISP-DM, por sus siglas en inglés) ofrece una referencia útil para organizar el trabajo en ciencia de datos: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue. Sin embargo, un proyecto de investigación no tiene que seguir estas fases de manera literal.

En una investigación académica, los investigadores pueden formular sus propias preguntas en lugar de traducir los objetivos de una organización cliente en tareas analíticas. Los conjuntos de datos disponibles pueden limitar las preguntas que es posible responder, y la exploración puede motivar revisiones del diseño original de la investigación. Por ello, el enfoque desarrollado aquí está **inspirado en CRISP-DM**, en lugar de ser una implementación estricta de CRISP-DM.

La metodología describe el proceso de investigación; la arquitectura del repositorio describe cómo documentar e implementar ese proceso. No es necesario que tengan estructuras idénticas. En particular, la organización de los cuadernos computacionales debe reflejar responsabilidades analíticas coherentes, no necesariamente la secuencia de fechas de entrega de informes.

2. Organización de un proyecto de investigación reproducible

2.1. El repositorio como unidad de reproducibilidad

Un solo cuaderno rara vez es suficiente para reproducir una investigación completa. La reproducción también puede requerir conjuntos de datos de entrada, código reutilizable, versiones de paquetes, archivos de configuración e instrucciones. Por lo tanto, el **repositorio** es la unidad natural para organizar los materiales reproducibles del proyecto.

El repositorio debe permitir determinar:

1. Qué insumos se utilizaron y de dónde provinieron.
2. Qué procedimientos generaron cada resultado analítico.
3. Qué productos se generaron y dónde se encuentran.
4. Qué supuestos y decisiones orientaron el análisis.
5. Cómo puede otra persona ejecutar los cálculos pertinentes.

El control de versiones, habitualmente con Git, registra la evolución de los archivos fuente y facilita la colaboración. Un repositorio por sí solo no garantiza la reproducibilidad: los datos externos, los entornos de software, las rutas de archivos y el orden de ejecución también requieren atención.

2.2. Una estructura mínima de directorios

Un proyecto pequeño puede comenzar sin directorios separados para funciones reutilizables o scripts de automatización:

```
project/
|-- README.md
|-- .gitignore
|-- renv.lock
|-- renv/
|   |-- activate.R
|-- data/
|   |-- raw/
|   |-- processed/
|-- notebooks/
|-- quarto/
|   |-- reports/
|   |-- _quarto.yaml
|-- docs/
    |-- reports/
```

Este ejemplo supone un proyecto de Quarto cuya raíz está en `project/quarto/`, con `project.output-dir: ../docs`; Quarto generalmente conserva los subdirectorios relativos a los archivos fuente dentro del directorio de salida. Debe verificarse la ubicación exacta de la salida para los formatos y la versión de Quarto seleccionados.

Ubicación	Responsabilidad
`README.md`	Explicar el proyecto, los requisitos previos y el procedimiento de reproducción

Ubicación	Responsabilidad
`quarto/_quarto.yaml`	Configurar la generación de documentos del proyecto de Quarto
`data/raw/`	Almacenar los datos de entrada originales sin modificaciones, cuando se permita su redistribución
`data/processed/`	Almacenar los datos procesados que se utilizarán en el proyecto de investigación
`notebooks/`	Almacenar documentos analíticos ejecutables
`quarto/reports/`	Almacenar los documentos fuente de los informes
`docs/`	Almacenar los productos generados que pueden distribuirse

Más adelante, un proyecto puede incorporar `src/` y `scripts/`. Estos directorios deben introducirse cuando resuelvan un problema organizativo real, no simplemente para imitar un proyecto de software de gran tamaño.

2.3. Archivos fuente y artefactos generados

Los **archivos fuente** son los materiales que editan los investigadores: documentos `.qmd`, código de R, configuración y documentación. Los **artefactos generados** son productos creados a partir de esas fuentes: informes en PDF o HTML, figuras, tablas y conjuntos de datos preparados.

Separar las fuentes de los productos generados reduce la ambigüedad. Un informe en PDF corresponde a una ubicación de publicación o distribución; el documento `.qmd` que lo genera corresponde a los materiales fuente editables del proyecto. Cuando sea práctico, los productos generados deben poder reproducirse a partir de las fuentes registradas en el control de versiones, en lugar de modificarse manualmente después de su generación.

No todos los archivos generados deben estar bajo seguimiento en Git. La decisión depende de si el archivo es un entregable obligatorio, de la facilidad con la que puede regenerarse, de su tamaño y de las restricciones aplicables al intercambio de datos. Los conjuntos de datos originales de gran tamaño o de carácter sensible pueden requerir un procedimiento de acceso en lugar de incluirse en el repositorio.

2.4. Nombres y rutas

Utilice nombres de archivo descriptivos y estables, como por ejemplo `data_quality.qmd` o `missingness_profile.qmd`. Los prefijos numéricos pueden ayudar cuando existe una secuen-

cia significativa, pero no deben implicar que los cuadernos requieran una sesión interactiva compartida.

Prefiera las rutas relativas al proyecto frente a las rutas absolutas específicas de una computadora. Un proyecto no debe depender del directorio personal de un estudiante en particular ni de archivos colocados manualmente fuera de la estructura documentada del repositorio. En R, herramientas como `here` pueden ayudar a construir rutas a partir de una raíz de proyecto consistente, siempre que dicha raíz esté configurada y se comprenda su función.

3. Cuadernos computacionales

3.1. La función de los cuadernos de Quarto

Un documento `.qmd` de Quarto combina prosa, código ejecutable de R y resultados computacionales. En esta guía, un **cuaderno** es un documento analítico ejecutable cuya responsabilidad principal es realizar y explicar una tarea computacional coherente.

Un cuaderno es más que una colección de comandos de R. Debe indicar qué pretende establecer, hacer explícitos sus insumos, documentar su procedimiento, presentar sus resultados y explicar qué implican esos resultados para la investigación.

3.2. Una responsabilidad analítica por cuaderno

Un principio útil, inspirado en la filosofía de Unix, es **hacer una sola cosa y hacerla bien**. Aplicado a los cuadernos, esto significa que cada documento debe tener un propósito analítico reconocible. Un cuaderno puede examinar la estructura de un conjunto de datos, evaluar la calidad de los datos o construir un conjunto de datos preparado. Cada tarea puede implicar varias funciones y cálculos.

La unidad apropiada de modularidad es una **tarea analítica coherente**, no una instrucción individual de R. Crear un cuaderno separado para cada cálculo menor introduciría una fragmentación innecesaria.

Un cuaderno puede reutilizarse o revisarse durante varios periodos de presentación de informes. A su vez, un solo informe puede apoyarse en varios cuadernos. Por lo tanto, los límites entre cuadernos no deben estar dictados únicamente por las fechas de entrega de los informes.

3.3. Anatomía estándar de un cuaderno

Una estructura interna consistente facilita la revisión y la reproducción de los cuadernos:

Elemento	Pregunta que aborda
Objetivo	¿Qué busca establecer o producir este cuaderno?
Insumos	¿Qué datos, parámetros y dependencias se requieren?
Procedimiento	¿Qué operaciones se realizan y en qué orden?
Productos	¿Qué tablas, figuras, conjuntos de datos o resultados se producen?
Interpretación	¿Qué significan los resultados y qué limitaciones persisten?

Esta es una plantilla conceptual, no una exigencia de utilizar exactamente cinco encabezados en cada documento. La explicación debe ser proporcional a la complejidad y la importancia de la tarea.

3.4. Ejecución independiente y dependencias explícitas

Una regla fundamental es que **cada cuaderno debe ejecutarse en una sesión nueva de R**, sin depender de objetos que hayan quedado en memoria de un trabajo interactivo previo. Puede depender de archivos producidos por otro paso, pero esas dependencias deben estar documentadas y ser reproducibles.

Por ejemplo, un cuaderno debe cargar su propio conjunto de datos de entrada en lugar de suponer que ya existe un objeto llamado `data` en el entorno global. Si se requiere un conjunto de datos procesado, el proyecto debe explicar cómo se crea ese conjunto de datos.

Este principio ayuda a prevenir un fallo común de los cuadernos: un documento que parece funcionar de manera interactiva, pero falla al ejecutarse de principio a fin. Una práctica útil de validación consiste en generar el documento del cuaderno en una sesión limpia e inspeccionar los productos y errores resultantes.

3.5. Transparencia y reproducibilidad computacionales

La **transparencia computacional** significa que el lector puede comprender qué cálculos se realizaron. La **reproducibilidad computacional** requiere, además, que esos cálculos puedan efectivamente ejecutarse de nuevo bajo condiciones documentadas.

El código legible y el texto explicativo favorecen la transparencia. Las dependencias explícitas, los insumos accesibles, la aleatoriedad controlada y la ejecución en una sesión limpia favorecen la reproducibilidad. Ninguna debe tratarse como sustituto de la otra.

4. Informes científicos

4.1. Los informes como comunicación científica

La responsabilidad principal de un informe es comunicar la investigación a un lector que quizá no examine el código. Debe explicar **qué** se investigó, **por qué** importa la pregunta, **qué** se encontró y **cómo deben interpretarse los hallazgos**.

El informe también debe describir los métodos analíticos con el nivel de detalle necesario para la evaluación científica. La distinción entre informes y cuadernos no consiste en que los informes omitan la metodología: más bien, los informes explican y justifican los métodos, mientras que los cuadernos muestran su implementación computacional y sus resultados intermedios.

Por ejemplo, un informe puede explicar por qué se excluyeron observaciones con valores faltantes y discutir las limitaciones resultantes. Un cuaderno de apoyo puede mostrar exactamente cómo se cuantificó la presencia de datos faltantes, qué filas se excluyeron y cómo se verificó el conjunto de datos resultante.

4.2. Informes y cuadernos como registros complementarios

Aspecto	Informe	Cuaderno
Pregunta de investigación	Enunciarla y explicar su motivación	Traducirla en tareas computacionales cuando corresponda
Decisiones metodológicas	Justificar las decisiones de mayor trascendencia	Implementarlas y documentarlas
Resultados	Presentar hallazgos seleccionados	Mostrar cálculos detallados y resultados intermedios
Interpretación	Explicar las implicaciones y limitaciones	Interpretar resultados diagnósticos específicos de la tarea
Reproducción	Remitir a los materiales de apoyo	Proporcionar evidencia ejecutable

La división es de **énfasis y público destinatario**, no una separación absoluta de los temas tratados. Las decisiones metodológicas significativas deben aparecer en el informe incluso cuando las operaciones correspondientes estén plenamente documentadas en los cuadernos.

4.3. Relación con los hitos de entrega de informes

Los informes pueden organizarse en torno a fechas de entrega, revisiones del proyecto o etapas de un curso. Los cuadernos computacionales no tienen que seguir ese mismo calendario. Un informe puede sintetizar varios cuadernos, y un cuaderno puede respaldar más de un informe.

Para preservar la trazabilidad, los informes deben identificar el cuaderno pertinente o la fuente reproducible de los principales hallazgos computacionales. Cuando sea viable, las tablas y figuras del informe deben generarse a partir de los mismos datos y cálculos subyacentes utilizados en los cuadernos, en lugar de transcribirse manualmente.

5. Flujos de trabajo modulares y automatizados

5.1. Modularidad analítica y modularidad del software

Deben distinguirse dos formas de modularidad:

- La **modularidad analítica** divide la investigación en tareas coherentes, cada una representada por un cuaderno u otro componente analítico bien definido.
- La **modularidad del software** separa la lógica computacional reutilizable en funciones y módulos que pueden invocarse desde distintos lugares.

La modularidad analítica puede aprenderse y practicarse antes que la modularidad del software. Es razonable que un proyecto pequeño contenga la mayor parte de su código directamente en los cuadernos. Los cálculos repetidos o complejos pueden extraerse posteriormente a funciones.

5.2. Funciones reutilizables de R y `src/`

A medida que crece un análisis, los cálculos o las transformaciones que se repiten pueden trasladarse a funciones de R almacenadas en `src/`. El cuaderno se centra entonces en la tarea analítica y su interpretación, mientras que la función encapsula una operación reutilizable.

Por ejemplo, un cálculo sencillo de la proporción de valores faltantes podría definirse así:

```
proportion_missing <- function(x) {
  mean(is.na(x))
}

proportion_missing(c(2, NA, 4, NA))
```

```
[1] 0.5
```

Este ejemplo ilustra la separación de responsabilidades, no una exigencia de convertir cada operación de una sola línea en una función. Extraer código vale la pena cuando mejora la claridad, la consistencia, las pruebas o la reutilización.

5.3. Scripts ejecutables y `scripts/`

Un directorio `scripts/` puede contener programas que coordinan tareas: adquirir datos, generar conjuntos de datos derivados, validar productos o generar una secuencia de documentos. Según esta convención, `src/` contiene componentes computacionales reutilizables, mientras que `scripts/` contiene puntos de entrada que ejecutan un flujo de trabajo.

Esta distinción es útil, pero no universal. Algunos proyectos utilizan otras convenciones de nombres o adoptan un gestor de flujos de trabajo. El requisito importante es que cada componente tenga una responsabilidad clara e insumos y productos documentados.

5.4. Del análisis interactivo a un proceso automatizado

Un proyecto puede desarrollarse progresivamente:

Nivel	Componentes principales	Capacidad principal
Análisis reproducible	Datos, cuadernos, informes	Volver a ejecutar análisis documentados
Cómputo modular	Incorporar funciones reutilizables de R	Reducir la duplicación y aislar la lógica computacional
Flujo de trabajo automatizado	Incorporar scripts de orquestación o herramientas de flujo de trabajo	Ejecutar operaciones vinculadas de manera sistemática

Estos niveles representan una sofisticación técnica creciente, no etapas obligatorias. Un proyecto de investigación pequeño puede ser plenamente adecuado en el primer nivel.

Un flujo de trabajo automatizado debe hacer explícitas las dependencias: qué insumos se requieren, qué proceso produce cada resultado y qué debe volver a ejecutarse cuando cambia un insumo. En proyectos complejos, las herramientas de flujo de trabajo pueden automatizar el seguimiento de dependencias; en proyectos más pequeños, pueden bastar instrucciones claras y unos cuantos scripts bien diseñados.

5.5. Verificaciones de reproducibilidad

Antes de considerar reproducible un análisis, verifique que:

1. Los insumos originales puedan identificarse y sean accesibles.
2. El proyecto explique cómo instalar o restaurar el entorno de software requerido.
3. Los cuadernos se ejecuten desde una sesión limpia en un orden documentado, o de manera independiente cuando así se haya previsto.
4. Las dependencias entre cuadernos, scripts y conjuntos de datos generados sean explícitas.
5. Los productos importantes puedan regenerarse sin modificaciones manuales no documentadas.
6. Los informes concuerden con los resultados computacionales que los respaldan.
7. Las decisiones metodológicas y las limitaciones estén registradas en una ubicación apropiada.

Estas verificaciones conectan las dimensiones computacional y metodológica de la reproducibilidad. La generación exitosa de un documento es una evidencia necesaria de reproducibilidad computacional, pero no establece por sí sola que el análisis haya sido metodológicamente apropiado.

6. Principios finales

Un proyecto reproducible de ciencia de datos se beneficia de una separación de responsabilidades:

- Los **informes** comunican el problema de investigación, justifican las decisiones metodológicas e interpretan los hallazgos.
- Los **cuadernos** realizan y explican tareas analíticas coherentes mediante documentación ejecutable.
- Las **funciones reutilizables** implementan operaciones computacionales que se benefician de ser compartidas.
- Los **scripts y las herramientas de flujo de trabajo** coordinan una ejecución repetible cuando el proyecto requiere automatización.
- El **repositorio** conecta estos materiales, sus insumos y sus productos en un conjunto comprensible y reproducible.

El objetivo no es imponer una arquitectura de software compleja a cada investigación. Es hacer que la investigación sea comprensible y que los cálculos sean reproducibles, incorporando estructura adicional solo cuando mejore la claridad, la fiabilidad o la facilidad de mantenimiento.